Computación gráfica y ambientes virtuales (0494)

Práctico 05

Objetivo

El objetivo es **poner una cámara en la escena**: reemplazar la constante `uAjuste` del *vertex shader* por una matriz de **vista** y una de **proyección** calculadas a partir de la elección de la visualización para la escena (cámara + tipo de proyección). La cámara es de tipo **orbital**, controlable mediante entradas del usuario por medio del mouse. El tipo de proyección para la cámara es **perspectiva**.

Punto de partida

La escena del Práctico 03 o la aeronave del Práctico 04, indistintamente: este práctico se enfoca en controlar los parámetros de visualización (cámara + tipo de proyección). Se puede utilizar la librería `glm` para la implementación —`glm::lookAt` y `glm::perspective` que se encuentran en `<glm/gtc/matrix_transform.hpp>` —. También hace falta `Shader` con sus métodos *setters* de *uniforms*.

El ajuste de la visualización hasta acá se hacía con una matriz "calculada a mano" y pasada como `uniform` al shader:

```
// solid.vs --- la versión actual
uniform mat4 uAjuste;   // "CAJA NEGRA"

// main.cpp --- la versión actual
const glm::mat4 m_ajuste = glm::scale(glm::mat4(1.0f),
                                      glm::vec3(600.0f/800.0f, 1.0f, -1.0f));
```

Este código **no es una proyección en perspectiva**: es un escalado que corrige la relación de aspecto de la ventana y da vuelta el eje z. Al ser un escalado, la cuarta fila de la matriz queda en $(0, 0, 0, 1)$: **no hay división por** w, y por eso un objeto que se aleja de la cámara **no se achica**.

Parte 1 — El módulo `CameraSystem`

Conviene que el cálculo de las matrices de visualización no quede suelto en el `main`: agruparlo en un módulo propio, por ejemplo `CameraSystem`, que sea el administrador del estado de la cámara en la escena. El resto del programa no necesita saber si la cámara es de órbita, de cabina o fija: sólo necesita, por cuadro, obtener **el par de matrices** para completar el proceso de proyección (vista -V- + proyección -P-). Esa separación es la que va a permitir, más adelante, agregar otros tipos de vistas sin tener que ajustar el resto del programa.

```
// CameraData.h --- punto de partida, no molde
struct CameraData {
    glm::mat4 view       = glm::mat4(1.0f);
    glm::mat4 projection = glm::mat4(1.0f);
};

// CameraCommand.h --- comandado por el usuario, en coordenadas esfericas
struct CameraCommand {
    float yaw_delta   = 0.0f;   // [rad] - cambio de longitud, acumulado en el cuadro
    float pitch_delta = 0.0f;   // [rad] - cambio de latitud
    float dist_delta  = 0.0f;   // [unidades de escena] - (+) se aleja
};

// CameraSystem.h --- punto de partida, no molde
class CameraSystem {
public:
    CameraSystem(int width, int height);            // arma la proyeccion inicial

    void update(glm::vec3 pos_avion,                // donde esta el objetivo
                glm::vec3 angulos_avion,            // no se usa por ahora (ver Parte 3)
                const CameraCommand& cmd);          // se actualiza en cada cuadro

    void set_viewport(int width, int height);       // al redimensionar la ventana

    const CameraData& data() const;                 // los datos para construir la proyección

private:
    float yaw_ = 0.0f, pitch_ = 0.0f, distancia_ = 10.0f;   // estado de la orbita
    CameraData data_ {};
};
```

Qué *no* hace este módulo.

- **No lee el mouse.** Recibe los cambios de posición realizados por el usuario ya interpretados como desplazamientos angulares/de distancia en `CameraCommand`. Se calculan en otro módulo.
- **No dibuja** por lo que no requiere nada de OpenGL.
- **No sabe cómo está compuesta la escena.** Por ahora sólo recibe un punto al que mirar.

Parte 2 — Actualizar la proyección (`set_viewport()`)

Primero hay que eliminar la "caja negra" del programa de shader para reflejar realmente como se quiere visualizar la escena. Esto implica dividir el uniform existente (`uAjuste`) en dos nuevos *uniforms* para poder recibir las matrices de transformación correspondientes y corregir el cálculo de la posición de los vértices que se procesan.

```
uniform mat4 uModel;
uniform mat4 uView;
uniform mat4 uProjection;
...
gl_Position = uProjection * uView * uModel * vec4(aPos, 1.0);
```

Luego, en el programa que administra la visualización (del lado del CPU) hay que definir el tipo y construir la proyección. En el caso de una *proyección en perspectiva* (y usando la librería `glm`) se necesitan los mismos cuatro parámetros vistos en la teoría de la Unidad VII:

`glm::perspective(fovy, aspect, near, far)`

- `fovy` es el ángulo de visión **vertical** θ_h, y va en **radianes** (`glm::radians(45.0f)`). Pasarlo en grados no genera ningún error: genera pantalla negra.
- `aspect` es la relación `ancho/alto` del *framebuffer*. Se usa para obtener el ángulo de visión horizontal: $\tan\frac{\theta_w}{2} = \text{aspect} \cdot \tan\frac{\theta_h}{2}$.

- `near` y `far` son las posiciones de los planos de recorte en unidades de la escena. Conviene **no** poner un valor de `near` muy chico: puede generar problemas de precisión al guardar la profundidad y, con ella, en la decisión de qué superficie ocluye a cuál (*z-buffer* — Unidad VIII).

Como `aspect` depende del tamaño de la ventana, entonces, la proyección hay que **recalcularla cada vez que la ventana achica o se agranda**. Eso responde a un *evento* por lo que se implementa mediante un *callback*:

```
void framebuffer_size_callback(GLFWwindow* window, int width, int height)
{
    glViewport(0, 0, width, height);      // el rectangulo de pixeles donde se dibuja
    camara.set_viewport(width, height);   // ... y recalcular la proyeccion:
}
```

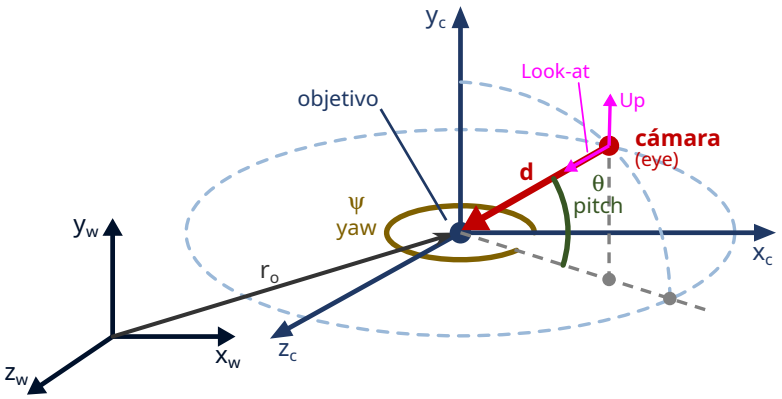
El tamaño del *framebuffer* lo cambia el sistema de ventanas, no OpenGL: la llamada a `glViewport` define el **rectángulo de ese *framebuffer*** sobre el que se mapean las coordenadas normalizadas, mientras que la segunda se encarga de recalcular la transformación de proyección para la nueva relación de aspecto. **Son dos cosas distintas y las dos hacen falta.**

Algunas cuestiones a pensar.

- Al minimizar la ventana, el alto puede llegar como **cero**. ¿Qué pasa con `aspect`? ¿Dónde conviene poner la comprobación que lo evita?
- El *callback* de GLFW tiene que ser una función, no un método: ¿cómo hace para llegar al objeto de la aplicación? (Ver `glfwSetWindowUserPointer` en la ayuda de GLFW.)

Parte 3 — La órbita y la matriz de vista (`update()`)

La cámara orbital se describe con **tres números** alrededor de un punto objetivo: dos ángulos y una distancia.



La cámara orbit: *yaw*, *pitch* y distancia al objetivo.

Los deltas que llegan en `CameraCommand` se **acumulan** sobre el estado interno, y la posición de la cámara sale de pasar esféricas a cartesianas:

$$x = d\cos(\theta)\cos(\psi) \qquad y = d\sin(\theta) \qquad z = d\cos(\theta)\sin(\psi)$$

$$\texttt{eye} = \texttt{objetivo} + (x,\, y,\, z) \quad \Longrightarrow \quad \texttt{view} = \texttt{glm::lookAt(eye, objetivo, up)}$$

Hay que acotar el estado: si el *pitch* llega a $\pm\pi/2$ la cámara queda justo sobre el eje vertical, entonces, `up` y la dirección de vista son **colineales** y el producto vectorial de adentro de `lookAt` se

hace cero dando como resultado una matriz indefinida. Acotarlo a, por ejemplo, $\pm 0{,}9 \cdot \pi/2$ es suficiente. La distancia también conviene acotarla entre un mínimo mayor que cero y un máximo para que la cámara no traspase los objetos por un lado y para no perderlos de la vista por otro.

Leer el mouse. Mover la cámara tiene que ser un **control continuo**, por lo que se determina el estado una vez por cuadro (estrategia de *polling*); no se espera un evento.

```
double x, y;
glfwGetCursorPos(window, &x, &y);   // POSICION, no desplazamiento

// el desplazamiento se calcula a mano:  delta = actual - anterior
// ... y despues se convierte a radianes usando una ganancia elegida (px -> rad)
```

Hay que controlar tres estados de la cámara con el mouse: rotar en acimut (yaw), rotar en elevación (pitch) y distancia al objeto. Como el movimiento que capta el mouse es bidimensional sólo se pueden asignar dos de los tres controles al mouse. Para poder controlar el tercero usando el mouse conviene programarlo para que uno de los desplazamientos del mouse (en X o en Y) cambie el tercer estado **sólo** mientras se encuentra presionado uno de los dos botones del mouse.

Algunas cuestiones a pensar.

- En el **primer cuadro** no hay "posición anterior". ¿Qué valor toma el desplazamiento? ¿Qué se ve en pantalla?
- La propuesta de `update()` recibe los **ángulos** de la aeronave, pero no son necesarios para la cámara orbital. ¿Para qué podrían servir cuando el avión se mueva? (Pista: pensar en una vista de cabina y en el sistema de referencia en el que estaría dada su posición.)
- El orden `uProjection * uView * uModel`: ¿qué se ve si se invierte? ¿Avisa el compilador?

Cómo se sabe que está bien.

- Un objeto que se aleja del observador **se achica**. Antes no pasaba: es la señal de que ahora sí hay perspectiva.
- Al arrastrar el mouse, la escena **queda quieta** y lo que se mueve es el punto de vista: el objeto no se deforma ni cambia de tamaño al girar alrededor.
- Al **redimensionar** la ventana arrastrando una esquina el modelo **no se deforma**: lo que era circular sigue siendo circular.
- Llevando el *pitch* hasta el tope la imagen **no da ningún salto** y no se pone en negro.

Como en los prácticos anteriores, las interfaces mostradas son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno, y hay que poder explicarlas.